



# PRS Pricing Model — Test Results

Automated Test Documentation for Model Governance

MKM Research Labs

Version 1.0 — 20-March-2026

David K Kelly

**CONFIDENTIAL — PROPRIETARY**

## **Legal Notice**

### **PROPRIETARY AND CONFIDENTIAL**

This document, including all algorithms, models, methodology, formulas, calculations, and intellectual concepts contained herein, constitutes the exclusive intellectual property and confidential trade secrets of MKM Research Labs (“MKM”).

Any usage, reproduction, distribution, modification, or disclosure of this document or any portion thereof, without the express prior written authorisation from MKM Research Labs is strictly prohibited and constitutes an infringement of intellectual property rights.

The document is provided on an “as is” basis. MKM Research Labs makes no representations or warranties of any kind, express or implied, regarding its accuracy, reliability, or suitability for any particular purpose.

All rights reserved. © 2021–2026 MKM Research Labs.

Governed by the laws of the United Kingdom.

# 1 Test Results — PRS Pricing Model (MKM-PR-001)

Tests run on **20 March 2026 at 12:15:19**.

Table 1: Test Results Summary — PRS Pricing Model (MKM-PR-001)

|             |       |
|-------------|-------|
| Total Tests | 61    |
| Passed      | 61    |
| Failed      | 0     |
| Skipped     | 0     |
| Duration    | 7.35s |

Table 2: Detailed Test Results — PRS Pricing Model

| Test Description                                 | Acceptance Criteria                                                                                 | Result | Time   |
|--------------------------------------------------|-----------------------------------------------------------------------------------------------------|--------|--------|
| Basis waterfall in generated output              | <code>bw1["model.uncertainty_bp"] == 2.0;</code><br><code>bw1["terrain_bp"] == 2.0 (+6 more)</code> | Pass   | 0.561s |
| Composition flat penalty                         | <code>result["composition_bp"] ==</code><br><code>COMPOSITION_BASIS.BPS["Flat"]</code>              | Pass   | 0.000s |
| Composition pre2000 penalty                      | <code>result["composition_bp"] == 1.0</code>                                                        | Pass   | 0.000s |
| Distance basis capped                            | <code>result["distance_bp"] ==</code><br><code>DISTANCE_MAX_BPS</code>                              | Pass   | 0.000s |
| Distance basis scales linearly                   | <code>result["distance_bp"] ==</code><br><code>pytest.approx(1.0, abs=0.1)</code>                   | Pass   | 0.000s |
| Elevation above gauge reduces spread             | <code>result["elevation_bp"] ==</code><br><code>pytest.approx(-1.0, abs=0.1)</code>                 | Pass   | 0.000s |
| Elevation below gauge no benefit                 | <code>result["elevation_bp"] == 0.0</code>                                                          | Pass   | 0.000s |
| Elevation capped at 3bp                          | <code>result["elevation_bp"] ==</code><br><code>-ELEVATION_MAX_BENEFIT_BPS</code>                   | Pass   | 0.000s |
| Model uncertainty is fixed                       | <code>result["model.uncertainty_bp"] ==</code><br><code>MODEL_UNCERTAINTY_BPS</code>                | Pass   | 0.000s |
| Terrain zone1 low risk                           | <code>result["terrain_bp"] == 0.0</code>                                                            | Pass   | 0.000s |
| Terrain zone3 high risk                          | <code>result["terrain_bp"] ==</code><br><code>TERRAIN_BASIS.BPS["Zone 3a"]</code>                   | Pass   | 0.000s |
| Total basis sums components                      | <code>result["total.basis_bp"] ==</code><br><code>pytest.approx(expected_total, ...)</code>         | Pass   | 0.000s |
| 1Y from Feb 2026 = last Friday May 2027.         | <code>mat == last_friday_of_month(2027, 5)</code>                                                   | Pass   | 0.000s |
| 3Y from Feb 2026 = last Friday May 2029.         | <code>mat == last_friday_of_month(2029, 5);</code><br><code>mat.month == 5 (+1 more)</code>         | Pass   | 0.000s |
| Jan 2027 uses Aug 2026 roll → Nov 2029 maturity. | <code>mat == last_friday_of_month(2029, 11)</code>                                                  | Pass   | 0.000s |

| Test Description                                                             | Acceptance Criteria                                                                                              | Result | Time   |
|------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|--------|--------|
| 3Y from Sep 2026 = last Friday Nov 2029 (Aug roll).                          | <code>mat == last_friday_of_month(2029, 11);</code><br><code>mat.month == 11</code>                              | Pass   | 0.000s |
| 5Y from Feb 2026 = last Friday May 2031.                                     | <code>mat == last_friday_of_month(2031, 5)</code>                                                                | Pass   | 0.000s |
| 5Y from Sep 2026 = last Friday Nov 2031.                                     | <code>mat == last_friday_of_month(2031, 11)</code>                                                               | Pass   | 0.000s |
| All maturities should fall on a Friday.                                      | <code>mat.weekday() == 4</code>                                                                                  | Pass   | 0.000s |
| 3Y maturity jumps from May to Nov at the Aug roll.                           | <code>may_mat.month == 5; nov_mat.month == 11 (+1 more)</code>                                                   | Pass   | 0.000s |
| Aug onwards uses Aug roll.                                                   | <code>roll == last_friday_of_month(2026, 8);</code><br><code>roll.month == 8</code>                              | Pass   | 0.000s |
| December uses the same year's Aug roll.                                      | <code>roll.month == 8; roll.year == 2026</code>                                                                  | Pass   | 0.000s |
| Feb-Jul uses Feb roll.                                                       | <code>roll == last_friday_of_month(2026, 2);</code><br><code>roll.month == 2</code>                              | Pass   | 0.000s |
| January uses the previous year's Aug roll.                                   | <code>roll == last_friday_of_month(2026, 8);</code><br><code>roll.month == 8 (+1 more)</code>                    | Pass   | 0.000s |
| July is the last month of the Feb roll period.                               | <code>roll.month == 2; roll.year == 2026</code>                                                                  | Pass   | 0.000s |
| March is still in the Feb roll period.                                       | <code>roll.month == 2; roll.year == 2026</code>                                                                  | Pass   | 0.000s |
| <code>compute_maturity_date</code> with no <code>ref_date</code> uses today. | <code>isinstance(mat, date); mat.weekday() == 4</code>                                                           | Pass   | 0.000s |
| <code>current_roll_date(None)</code> uses today and returns a valid date.    | <code>isinstance(roll, date);</code><br><code>roll.weekday() == 4</code>                                         | Pass   | 0.000s |
| <code>maturity_table</code> with no <code>ref_date</code> uses today.        | <code>len(table) == 5;</code><br><code>all(isinstance(e['maturity_date'], date) for e in table)</code>           | Pass   | 0.000s |
| Empty propertyts dir                                                         | <code>stats["total_properties"] == 0;</code><br><code>stats["properties_with_gev"] == 0</code>                   | Pass   | 0.002s |
| Generator should still work without gauge hazard curves (empty basis).       | <code>stats["properties_with_gev"] == 2;</code><br><code>stats["properties_with_floor"] == 1</code><br>(+1 more) | Pass   | 0.564s |
| No propertyts dir                                                            | —                                                                                                                | Pass   | 0.000s |
| Aug 2026: 31st is Monday, last Friday = 28th.                                | <code>last_friday_of_month(2026, 8) == date(2026, 8, 28)</code>                                                  | Pass   | 0.000s |
| Feb 2026: 28th is Saturday, last Friday = 27th.                              | <code>last_friday_of_month(2026, 2) == date(2026, 2, 27)</code>                                                  | Pass   | 0.000s |
| May 2029: 31st is Thursday, last Friday = 25th.                              | <code>last_friday_of_month(2029, 5) == date(2029, 5, 25)</code>                                                  | Pass   | 0.000s |

| Test Description                                       | Acceptance Criteria                                                                                                            | Result | Time   |
|--------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|--------|--------|
| Nov 2029: 30th is Friday.                              | <code>last_friday_of_month(2029, 11) == date(2029, 11, 30)</code>                                                              | Pass   | 0.000s |
| Result should always be a Friday (weekday=4).          | <code>d.weekday() == 4</code>                                                                                                  | Pass   | 0.000s |
| Actual years should be strictly increasing.            | <code>years[i] &gt; years[i - 1]</code>                                                                                        | Pass   | 0.000s |
| All maturities should be in May for the Feb roll.      | <code>e['maturity_date'].month == 5</code>                                                                                     | Pass   | 0.000s |
| All maturities should be in November for the Aug roll. | <code>e['maturity_date'].month == 11</code>                                                                                    | Pass   | 0.000s |
| Maturity string should be formatted as 'DD Mon YYYY'.  | <code>len(e['maturity_str']) &gt; 8;</code> 'May' in <code>e['maturity_str']</code> or 'Nov' in <code>e['maturity_str']</code> | Pass   | 0.000s |
| Default table has 5 entries (1Y through 5Y).           | <code>len(table) == 5</code>                                                                                                   | Pass   | 0.000s |
| Tenors should be 1, 2, 3, 4, 5.                        | <code>tenors == [1, 2, 3, 4, 5]</code>                                                                                         | Pass   | 0.000s |
| Positive spread                                        | <code>spread &gt; 0; 50 &lt; spread &lt; 500</code>                                                                            | Pass   | 0.000s |
| Spread increases with hazard                           | <code>spread_high &gt; spread_low</code>                                                                                       | Pass   | 0.000s |
| Spread values at different tenors                      | <code>s &gt; 0</code>                                                                                                          | Pass   | 0.000s |
| Zero hazard rate                                       | <code>spread == 0.0</code>                                                                                                     | Pass   | 0.000s |
| Basis calculation                                      | <code>len(nearest) &gt; 0;</code> <code>ng0["gauge_id"] == "GAUGE-001"</code> (+5 more)                                        | Pass   | 0.553s |
| Depth thresholds                                       | <code>"any_flood" in thresholds;</code> <code>"moderate" in thresholds</code> (+4 more)                                        | Pass   | 0.560s |
| Generate produces output                               | <code>output_path.exists();</code> <code>"metadata" in data</code> (+2 more)                                                   | Pass   | 0.956s |
| Gev params structure                                   | <code>"shape" in gev;</code> <code>"loc" in gev</code> (+2 more)                                                               | Pass   | 0.527s |
| High transmission rate                                 | <code>prop3["summary"]["flood.transmission_rate"] == 1.0</code>                                                                | Pass   | 0.544s |
| Output structure                                       | <code>"PROP-001" in curves;</code> <code>"PROP-002" in curves</code> (+21 more)                                                | Pass   | 0.546s |
| Properties with gev                                    | <code>stats["properties_with_gev"] == 2;</code> <code>stats["properties_with_floor"] == 1</code> (+1 more)                     | Pass   | 0.703s |
| Survival decreases with tenor                          | <code>survival[i] &lt;= survival[i - 1]</code>                                                                                 | Pass   | 0.598s |
| Term structure                                         | <code>ts["tenors"] == TENORS;</code> <code>threshold_name in ts</code> (+3 more)                                               | Pass   | 0.631s |
| Recovery rates by trigger                              | <code>len(spreads) &gt; 0</code>                                                                                               | Pass   | 0.603s |
| Recovery reduces spread                                | <code>spread_with_recovery &lt; spread_no_recovery</code>                                                                      | Pass   | 0.000s |

| Test Description                                        | Acceptance Criteria                                           | Result | Time   |
|---------------------------------------------------------|---------------------------------------------------------------|--------|--------|
| All tenors should round-trip through maturity and back. | <code>tenor_from_maturity(mat, ref) == t</code>               | Pass   | 0.000s |
| A date not in the table returns the closest tenor.      | <code>result in [1, 2, 3, 4, 5]</code>                        | Pass   | 0.000s |
| Exact maturity date returns correct tenor.              | <code>tenor_from_maturity(mat, date(2026, 2, 24)) == 3</code> | Pass   | 0.000s |

## 1.1 Test Document History

| Date        | Version | Description                                      | Author   |
|-------------|---------|--------------------------------------------------|----------|
| 05-Mar-2026 | 1.0     | Initial test suite (30 tests)                    | D. Kelly |
| 07-Mar-2026 | 1.1     | 57 test(s) added; 30 test(s) removed (total: 57) | D. Kelly |
| 07-Mar-2026 | 1.2     | 30 test(s) added (total: 87)                     | D. Kelly |
| 18-Mar-2026 | 1.3     | 4 test(s) added (total: 91)                      | D. Kelly |
| 20-Mar-2026 | 1.4     | 30 test(s) removed (total: 61)                   | D. Kelly |